LangChain

DeepLearning.AI

Short Course Template
8 wide by 9 high

AI Agents in LangGraph

Harrison Chase
Rotem Weiss







AI Agents in LangGraph

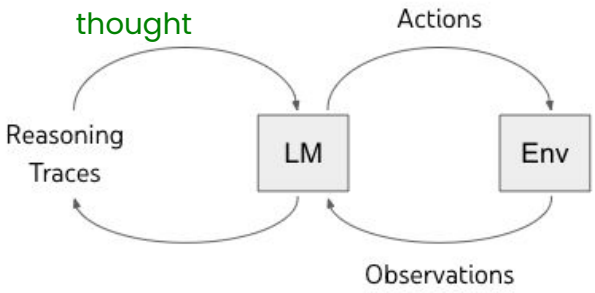
Build an Agent from Scratch



LangChain

DeepLearning.AI

Let's build an agent from scratch



ReAct (Reason + Act)

Published as a conference paper at ICLR 2023

REACT: SYNERGIZING REASONING AND ACTING IN LANGUAGE MODELS

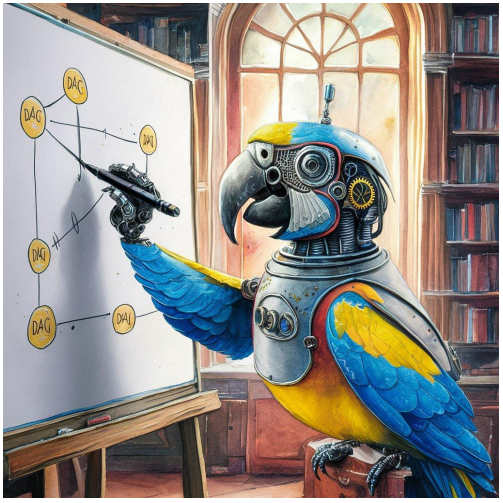
Shunyu Yao*,[1], Jeffrey Zhao[2], Dian Yu[2], Nan Du[2], Izhak Shafran[2], Karthik Narasimhan[1], Yuan Cao[2]

[1]Department of Computer Science, Princeton University
[2]Google Research, Brain team
[1]{shunyuy,karthikn}@princeton.edu
[2]{jeffreyzhao,dianyu,dunan,izhak,yuancao}@google.com

https://arxiv.org/pdf/2210.03629.pdf

AI Agents in LangGraph

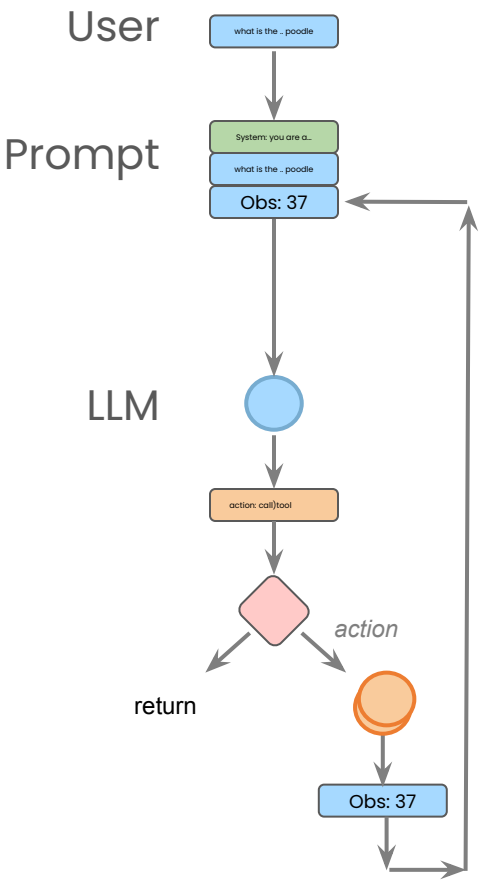
LangGraph Components



LangChain

DeepLearning.AI

Break Down



“””
System:
You run in a loop of Thought, Action, PAUSE, Observation.
…
Your available actions are:

calculate:
e.g. calculate: 4 * 7 / 3
…
Example session:
…
User:
… weight of collie…

'Thought: To find the combined weight of a Border Collie and a Scottish Terrier, I need to first find the average weight of each breed and then add those weights together.
Action: average_dog_weight: Border Collie\n PAUSE'

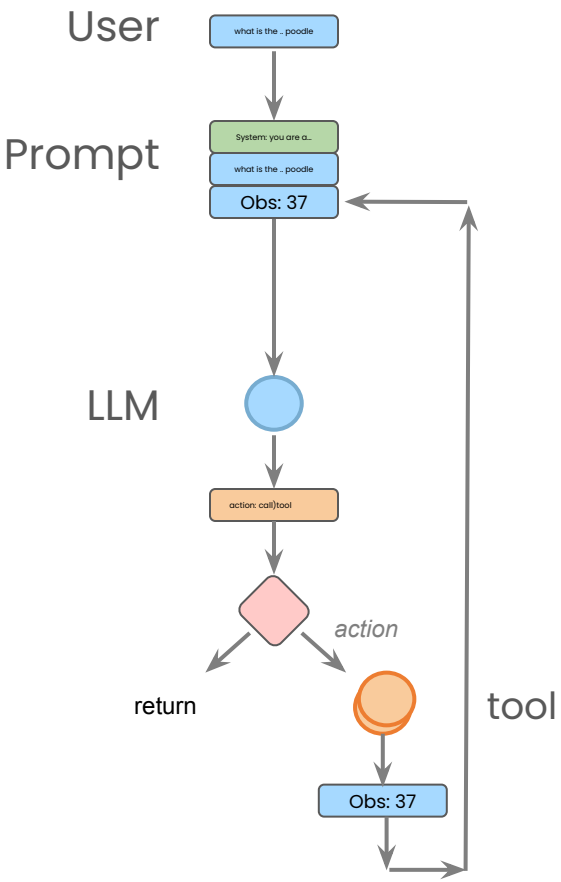
Observation: a Border Collies average weight is 37 lbs

```
def query(question, max_turns=5):
    i = 0
    bot = Agent(prompt)
    next_prompt = question
    while i < max_turns:
        i += 1
        result = bot(next_prompt)
        print(result)
        actions = [action_re.match(a) for a in result.split('\n') if action_re.match(a)]
        if actions:
            # There is an action to run
            action, action_input = actions[0].groups()
            if action not in known_actions:
                raise Exception("Unknown action: {}: {}".format(action, action_input))
            print(" -- running {} {}".format(action, action_input))
            observation = known_actions[action](action_input)
            print("Observation:", observation)
            next_prompt = "Observation: {}".format(observation)
        else:
            return
```

```
def calculate(what):
    return eval(what)

def average_dog_weight(name):
    if name in "Scottish Terrier":
        return("Scottish Terriers average 20 lbs")
    elif name in "Border Collie":
        return("a Border Collies average weight is 37
lbs")
    elif name in "Toy Poodle":
        return("a toy poodles average weight is 7 lbs")
    else:
        return("An average dog weights 50 lbs")
```

LangChain: Prompts



Prompt templates allow reusable prompts

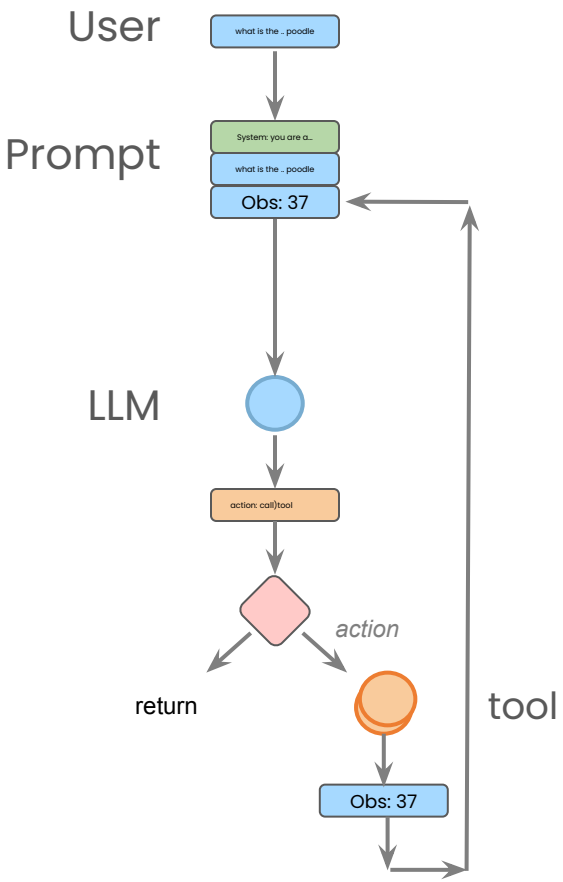
```
from langchain.prompts import PromptTemplate
prompt_template = PromptTemplate.from_template(
  "Tell me a {adjective} joke about {content}."
```

There are also prompts for agents available in the hub:

```
prompt = hub.pull("hwchase17/react")
```

https://smith.langchain.com/hub/hwchase17/react

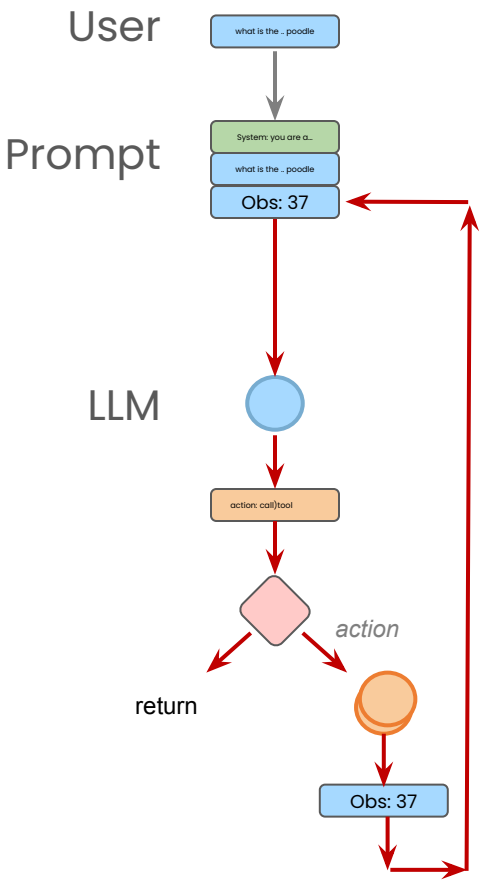
LangChain: Tools




```
# get a tool from the library
from langchain_community.tools.tavily_search import \
TavilySearchResults
tool = TavilySearchResults(max_results=2)

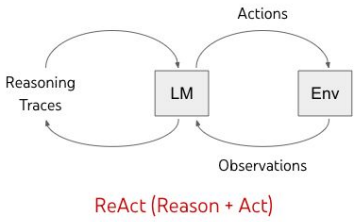
self.tools = {t.name: t for t in tools}
self.model = model.bind_tools([tool])
```

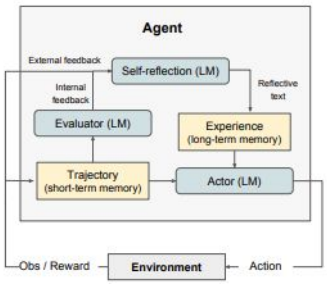
New in LangGraph

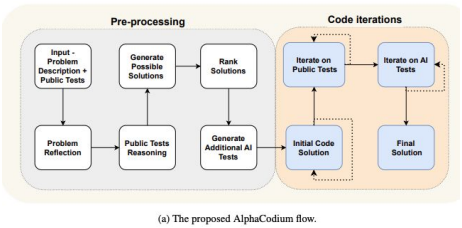


- Cyclic Graphs
- Persistence
- Human-in-the-loop

Graphs







(a) The proposed AlphaCodium flow.

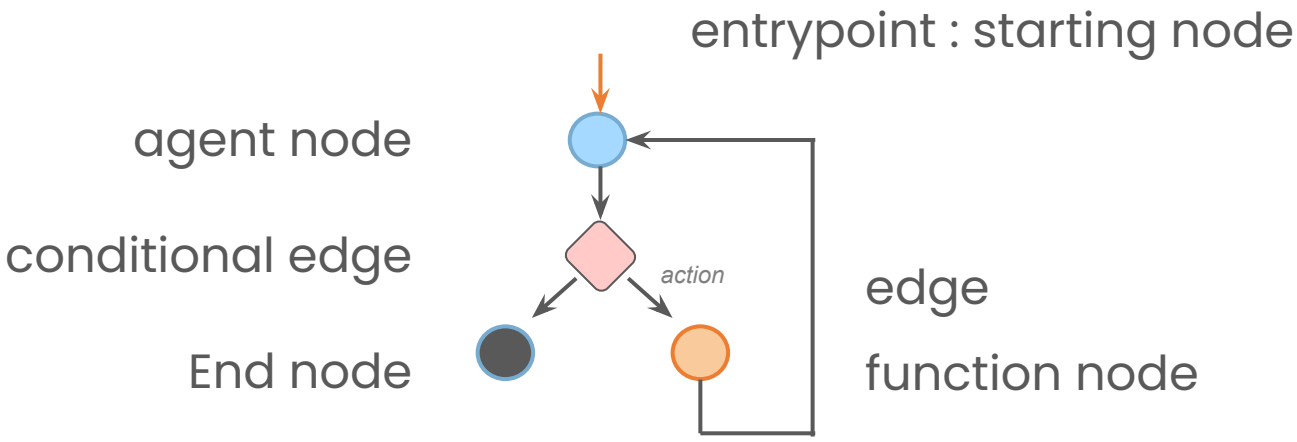
- LangGraph is an extension of LangChain that supports graphs.
- Single and Multi-agent flows are described and represented as graphs.
- Allows for extremely controlled “flows”
- Built-in persistence allows for human-in-the-loop workflows

Graphs

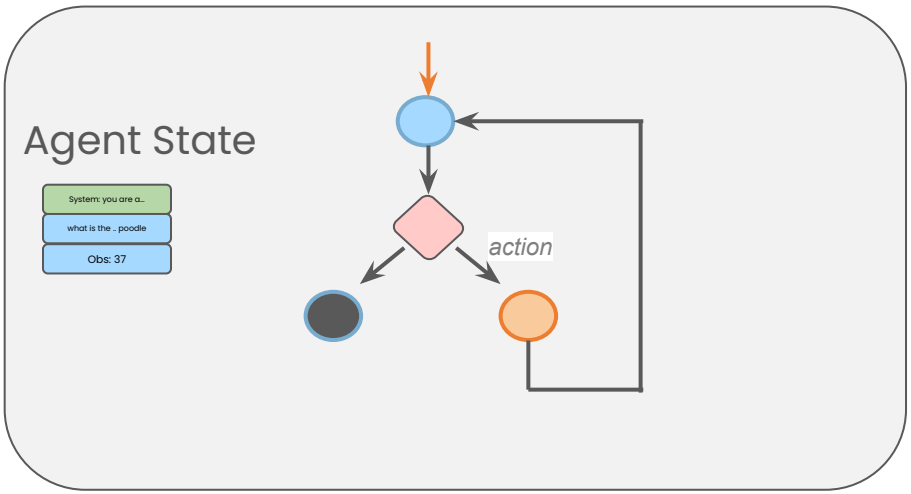
Nodes: Agents or functions

Edges: connect nodes

Conditional edges: decisions



Data/State



- Agent State is accessible to all parts of the graph
- It is local to the graph
- Can be stored in a persistence layer

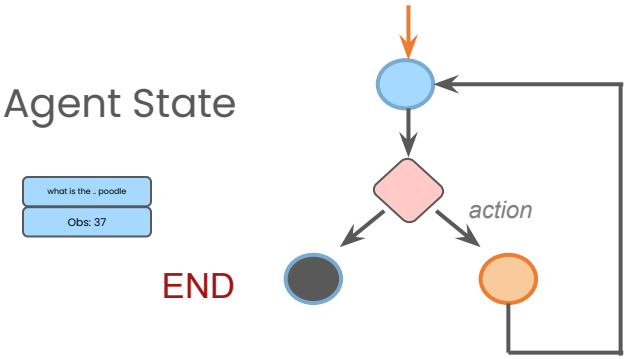
Simple

```
class AgentState(TypedDict):
    messages: Annotated[Sequence[BaseMessage], operator.add]
```

Complex

```
class AgentState(TypedDict):
    input: str
    chat_history: list[BaseMessage]
    agent_outcome: Union[AgentAction, AgentFinish, None]
    intermediate_steps: Annotated[list[tuple[AgentAction, str]], operator.add]
```

CODE



llm: call_openai

c_edge: exists_action

action:take_action

State

```
class AgentState(TypedDict):
    messages: Annotated[list[AnyMessage], operator.add]
```

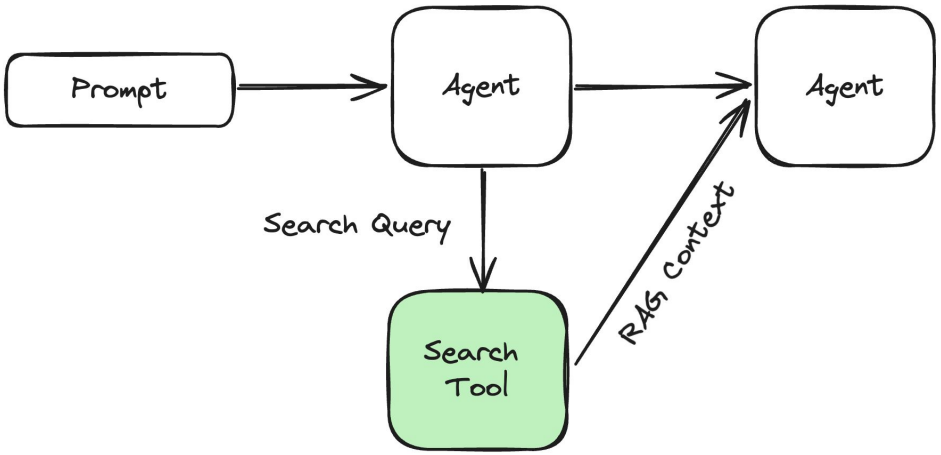
AI Agents in LangGraph

Agentic Search Tools

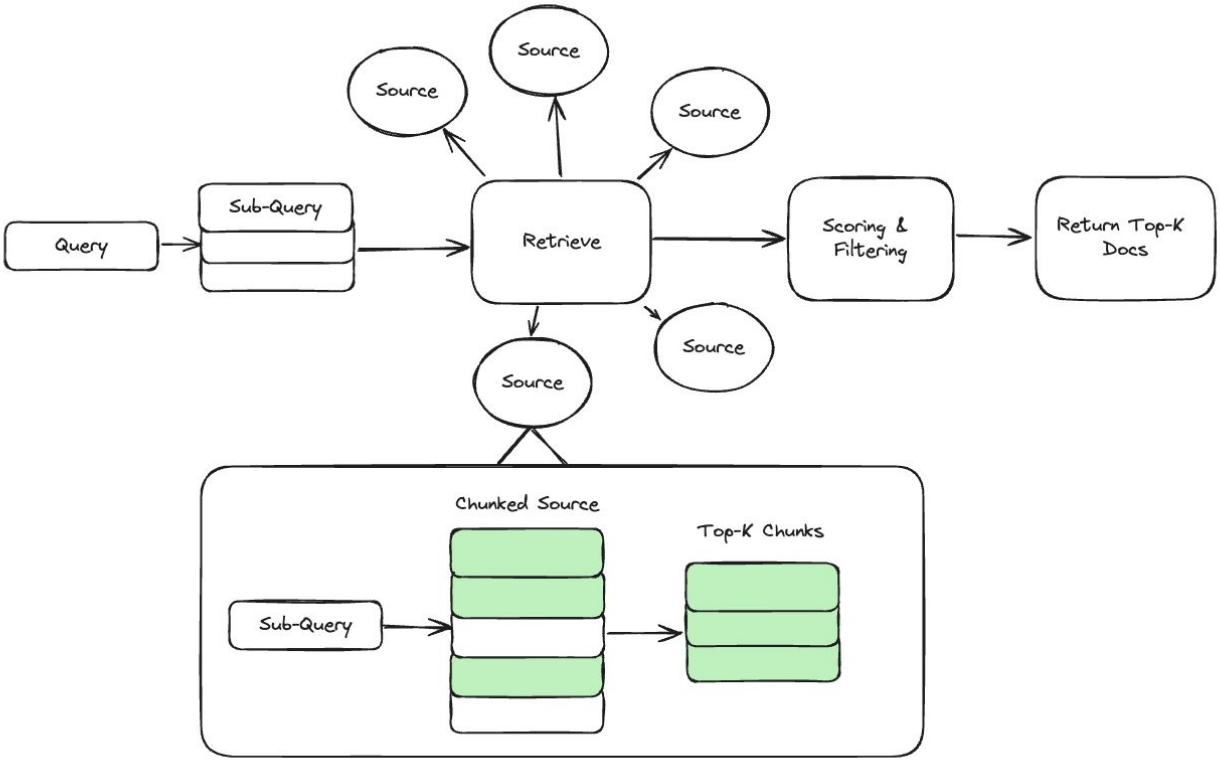


LangChain Tavily DeepLearning.AI

Why Search Tool



Inside a Search Tool



AI Agents in LangGraph

Persistence and Streaming



LangChain

DeepLearning.AI

AI Agents in LangGraph

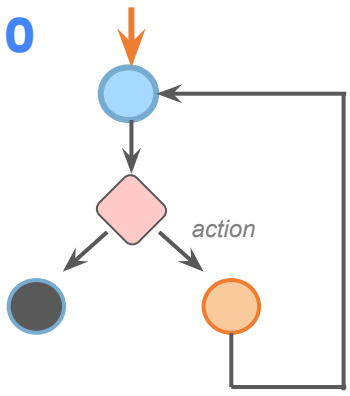
Human in the loop



LangChain

DeepLearning.AI

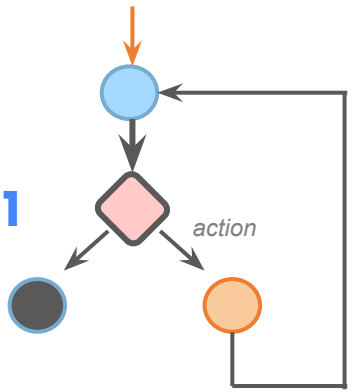
State Memory



Memory

state 0

State Memory

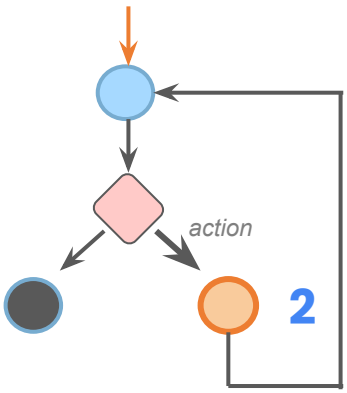


Memory

state 1

state 0

State Memory



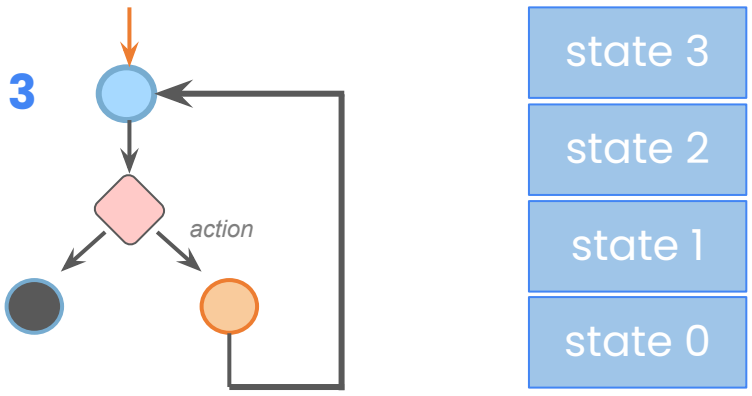
Memory

state 2

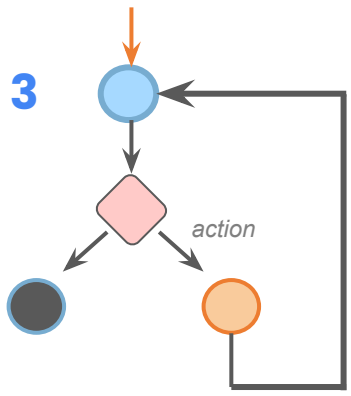
state 1

state 0

State Memory



State Memory

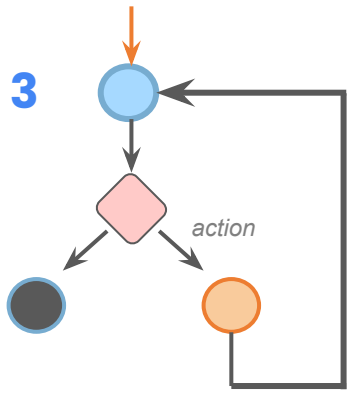


Memory



StateSnapshot: { AgentState, useful_things}

State Memory



Memory

state 3

state 2

state 1

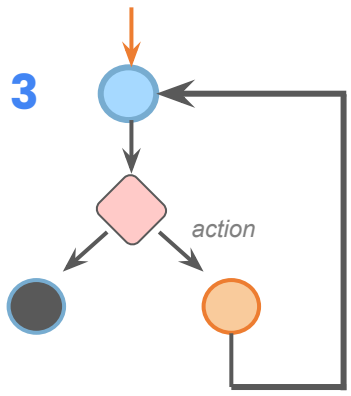
state 0

StateSnapshot: { AgentState, useful_things}

{ .., thread, thread_ts..}

```
config={'configurable': {'thread_id': '1',
                         'thread_ts': '1ef17b36-ed06-6185-8001-15cf75dea535'}}
```

State Memory



Memory

state 3

state 2

state 1

state 0

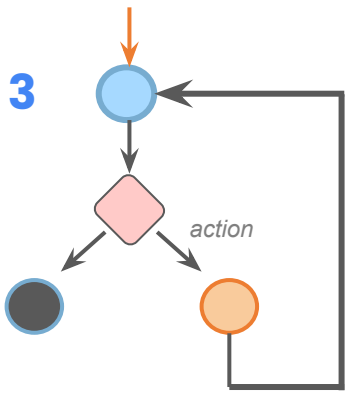
config={'configurable': {'thread_id': '1'}}

g.get_state(thread)

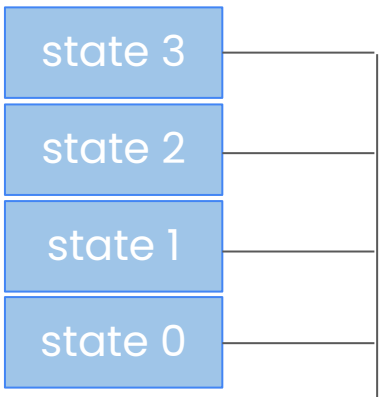
state 3

Returns ‘current state’

State Memory



Memory

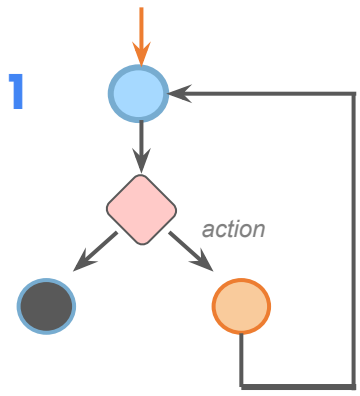


g.get_state_history(thread)

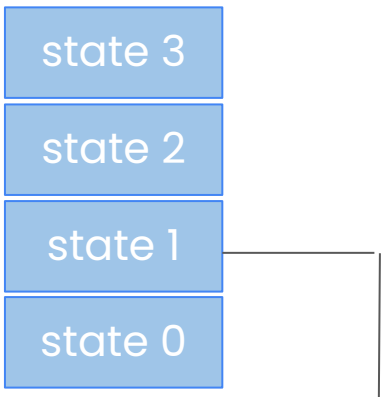


Returns iterator over all StateSnapshots

State Memory



Memory

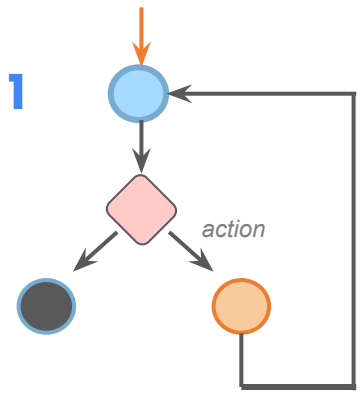


g.invoke(None,{ .., thread, thread_ts..})
g.stream(None,{ .., thread, thread_ts..})

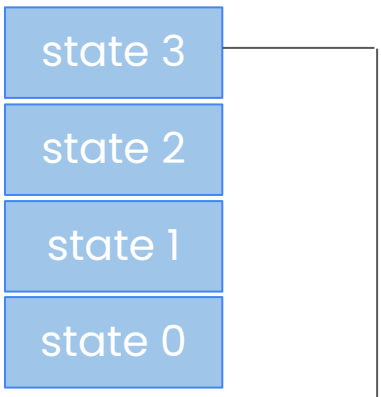
state 1

runs with state 1 as starting point
Time Travel

State Memory



Memory



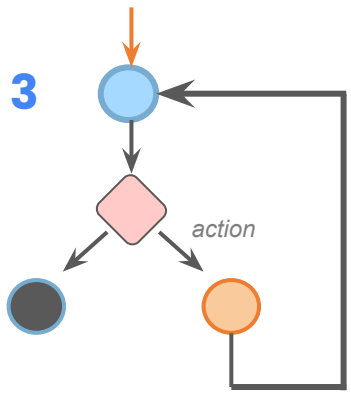
without thread_ts

```
g.invoke(None,{ .., thread})
g.stream(None,{ .., thread})
```

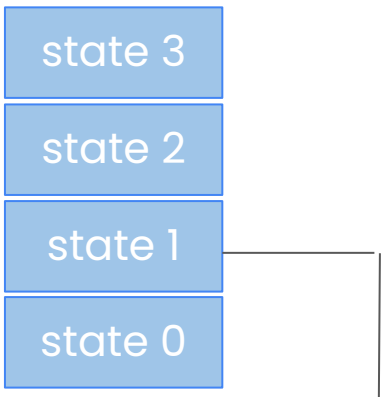


Uses current state as starting point

State Memory



Memory

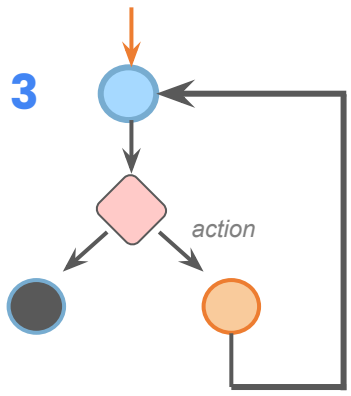


g.get_state({thread, thread_ts})

state 1

Returns 'thread_ts state-snapshot'

State Memory



Memory

state 3

state 2

state 1

state 0

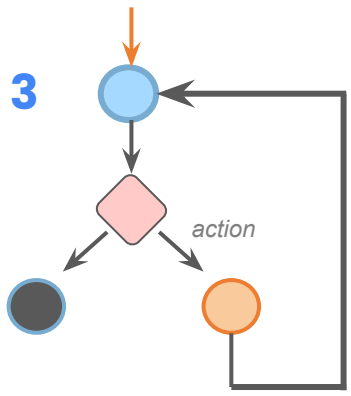
g.get_state({thread, thread_ts})

state 1

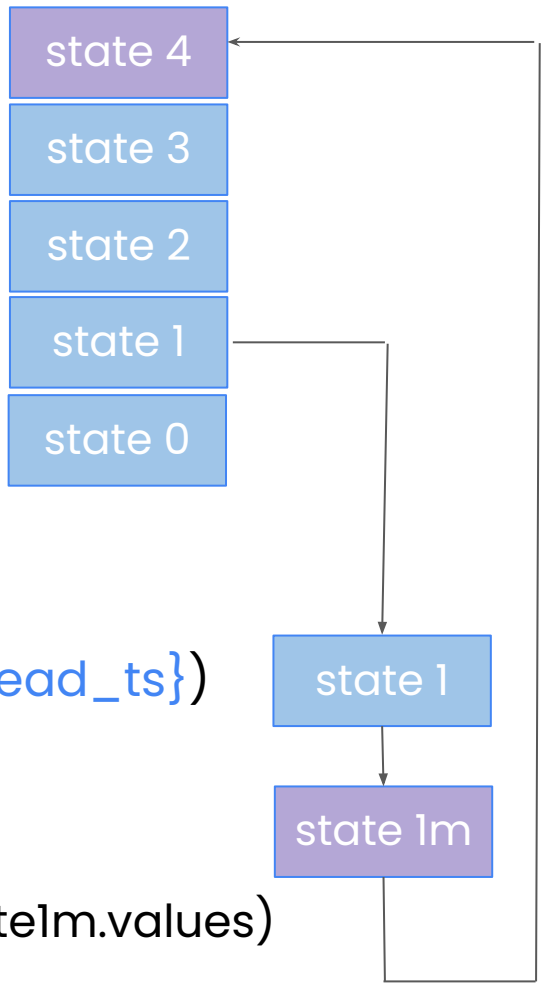
modify it

state 1m

State Memory



Memory

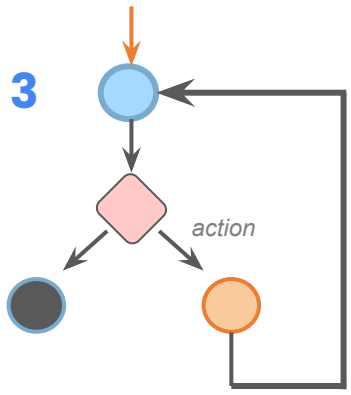


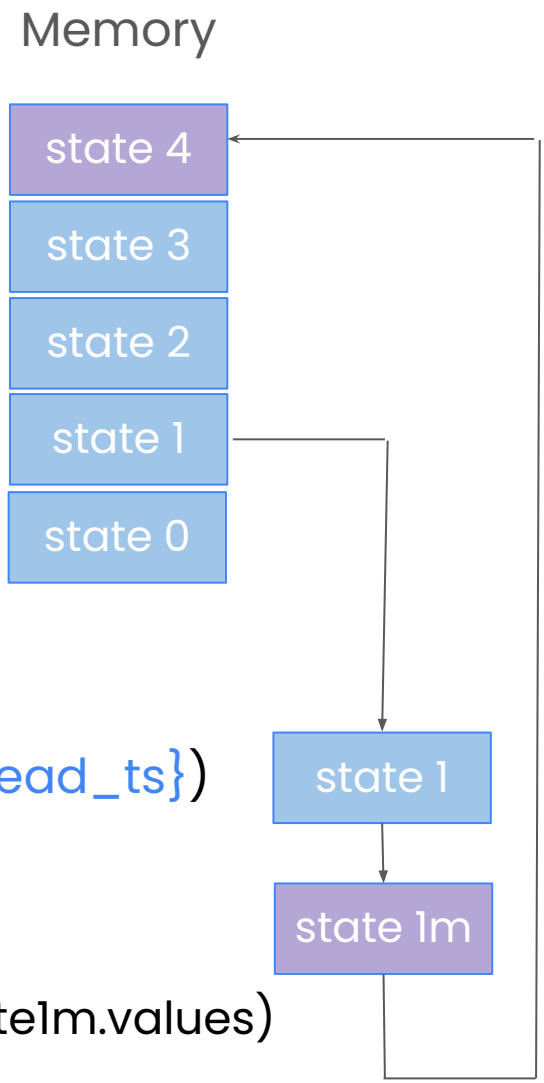
g.get_state({thread, thread_ts})

modify it

g.update_state(thread, state1m.values)

State Memory





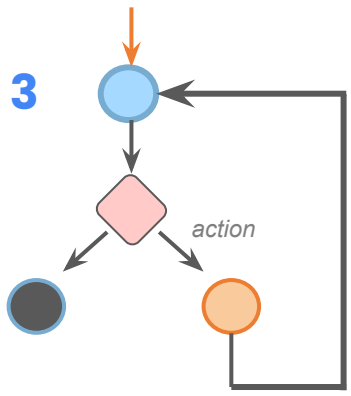
g.get_state({thread, thread_ts})

modify it

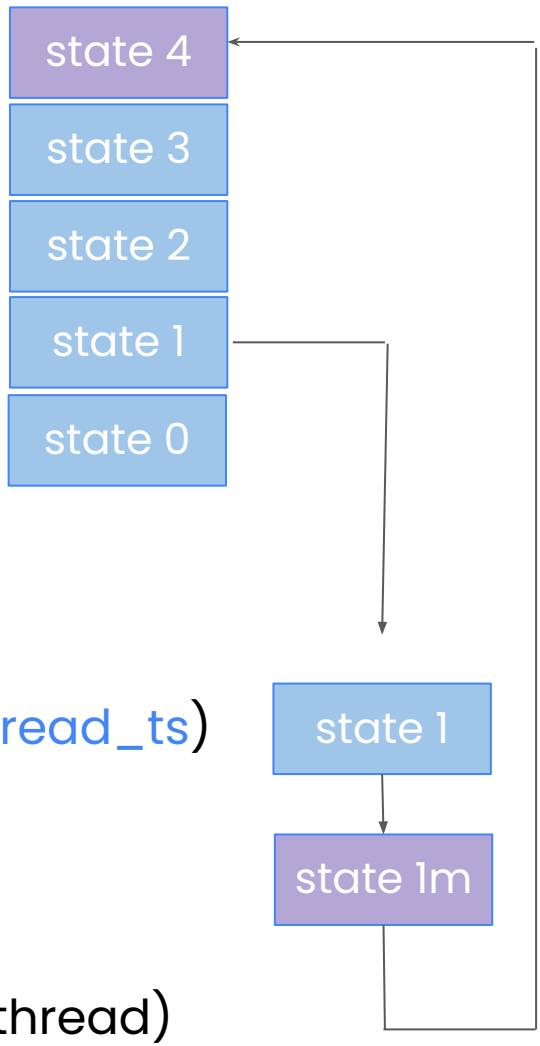
g.update_state(thread, state1m.values)

run it g.stream(None, thread)

State Memory



Memory



g.get_state(thread, thread_ts)

modify it

g.stream(state 1m, thread)

AI Agents in LangGraph

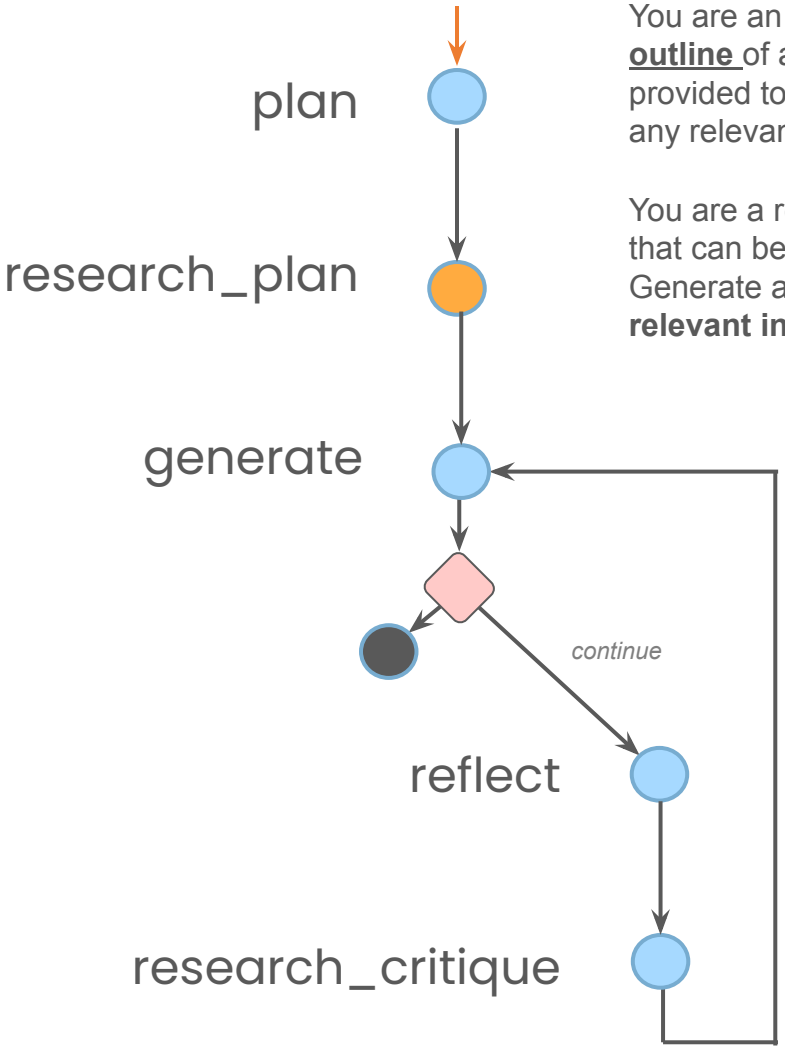
Essay Writer



LangChain

DeepLearning.AI

Essay Writer



You are an expert writer tasked with writing a **high level outline** of an essay. Write such an outline for the user provided topic. Give an outline of the essay along with any relevant notes or instructions for the sections.

You are a researcher charged with providing information that can be used when writing the following essay. Generate a list of **search queries that will gather any relevant information.** Only generate 3 queries max.

You are an essay assistant tasked with **writing excellent 5-paragraph essays.**Generate … If the **user provides critique, respond with a revised version** of your previous attempts. Utilize information:

Plan:

I. Introduction
   A. Brief overview of Langchain and Langsmith
   B. Thesis statement: Exploring the key differences between Langchain and Langsmith

II. Langchain
   A. Definition and purpose
   B. Technology and applications
   C. Key features and benefits

III. Langsmith
   A. Definition and purpose
   B. Technology and applications
   C. Key features and benefits

IV. Comparison between Langchain and Langsmith
   A. Technology stack
   B. Use cases and industries
   C. Scalability and performance
   D. Security and privacy considerations

V. Conclusion
   A. Recap of main differences between Langchain and Langsmith
   B. Future implications and potential developments in the field

Notes:
- Ensure to provide clear definitions and examples for both Langchain and Langsmith.
- Compare and contrast the technologies in terms of their features, applications, and benefits.
- Include real-world examples or case studies to illustrate the differences between Langchain and Langsmith.
- Conclude by summarizing the key points and discussing the potential impact of these technologies on the industry.

AI Agents in LangGraph

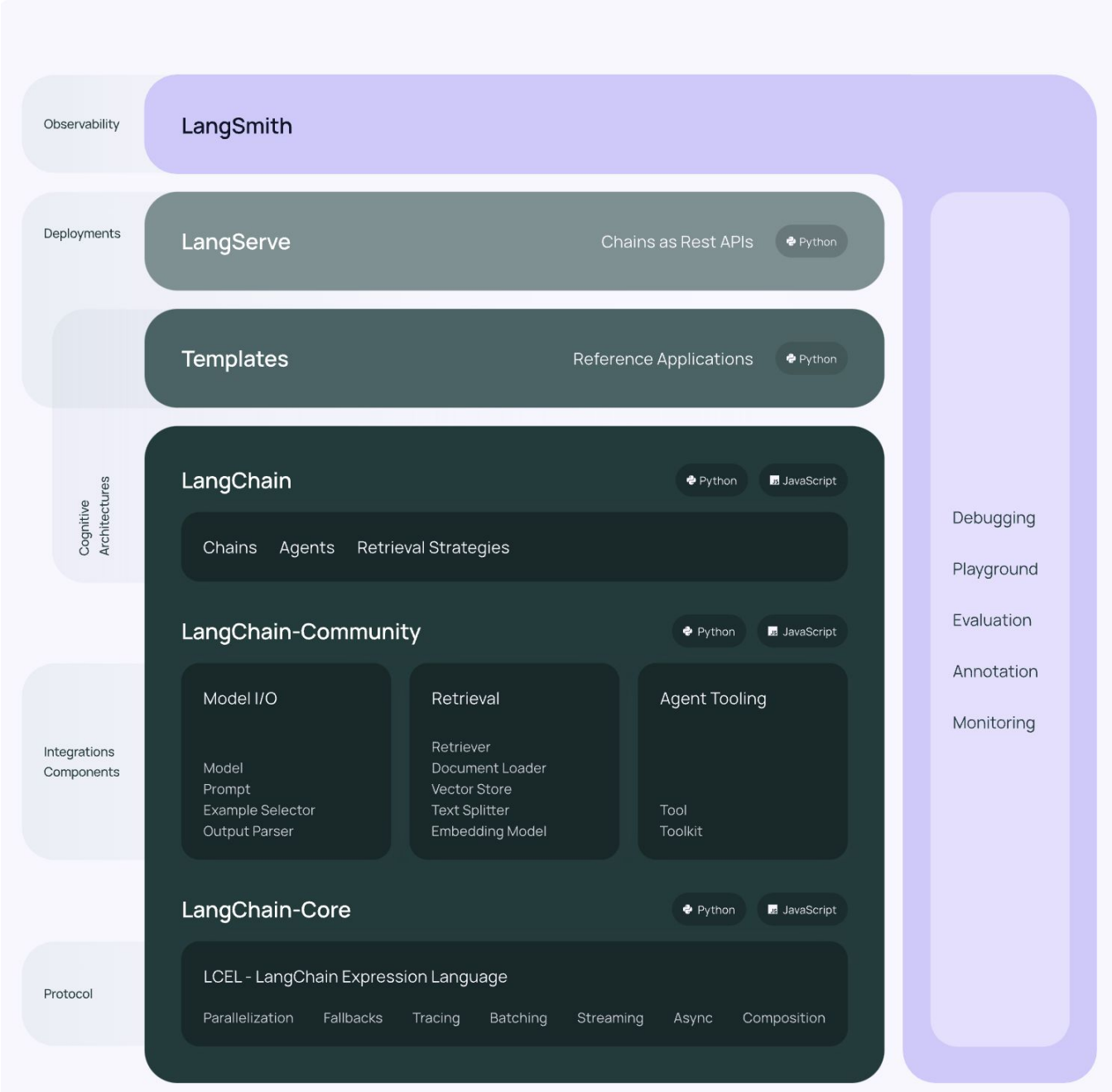
LangChain Resources



LangChain

DeepLearning.AI

Guide


Observability
LangSmith
Deployments
LangServe
Chains as Rest APIs
Python
Templates
Reference Applications
Python
Cognitive Architectures
LangChain
Python
JavaScript
Chains
Agents
Retrieval Strategies
LangChain-Community
Python
JavaScript
Integrations Components
Model I/O
Model
Prompt
Example Selector
Output Parser
Retrieval
Retriever
Document Loader
Vector Store
Text Splitter
Embedding Model
Agent Tooling
Tool
Toolkit
LangChain-Core
Python
JavaScript
Protocol
LCEL - LangChain Expression Language
Parallelization
Fallbacks
Tracing
Batching
Streaming
Async
Composition
Debugging
Playground
Evaluation
Annotation
Monitoring

Organization

Concretely, the framework consists of the following open-source libraries:

- `langchain-core`: Base abstractions and LangChain Expression Language.
- `langchain-community`: Third party integrations.
  - Partner packages (e.g. `langchain-openai`, `langchain-anthropic`, etc.): Some integrations have been further split into their own lightweight packages that only depend on `langchain-core`.
- `langchain`: Chains, agents, and retrieval strategies that make up an application's cognitive architecture.
- langgraph: Build robust and stateful multi-actor applications with LLMs by modeling steps as edges and nodes in a graph.
- langserve: Deploy LangChain chains as REST APIs.

The broader ecosystem includes:

- LangSmith: A developer platform that lets you debug, test, evaluate, and monitor LLM applications and seamlessly integrates with LangChain.

> Describe when to use LangGraph vs LangChains agents

Resources

- github Examples:
  - https://github.com/langchain-ai/langchain
- Hub
- Docs
- GitHub LangGraph
  - Descriptions and pointers to examples
  - https://github.com/langchain-ai/langgraph/tree/main

AI Agents in LangGraph

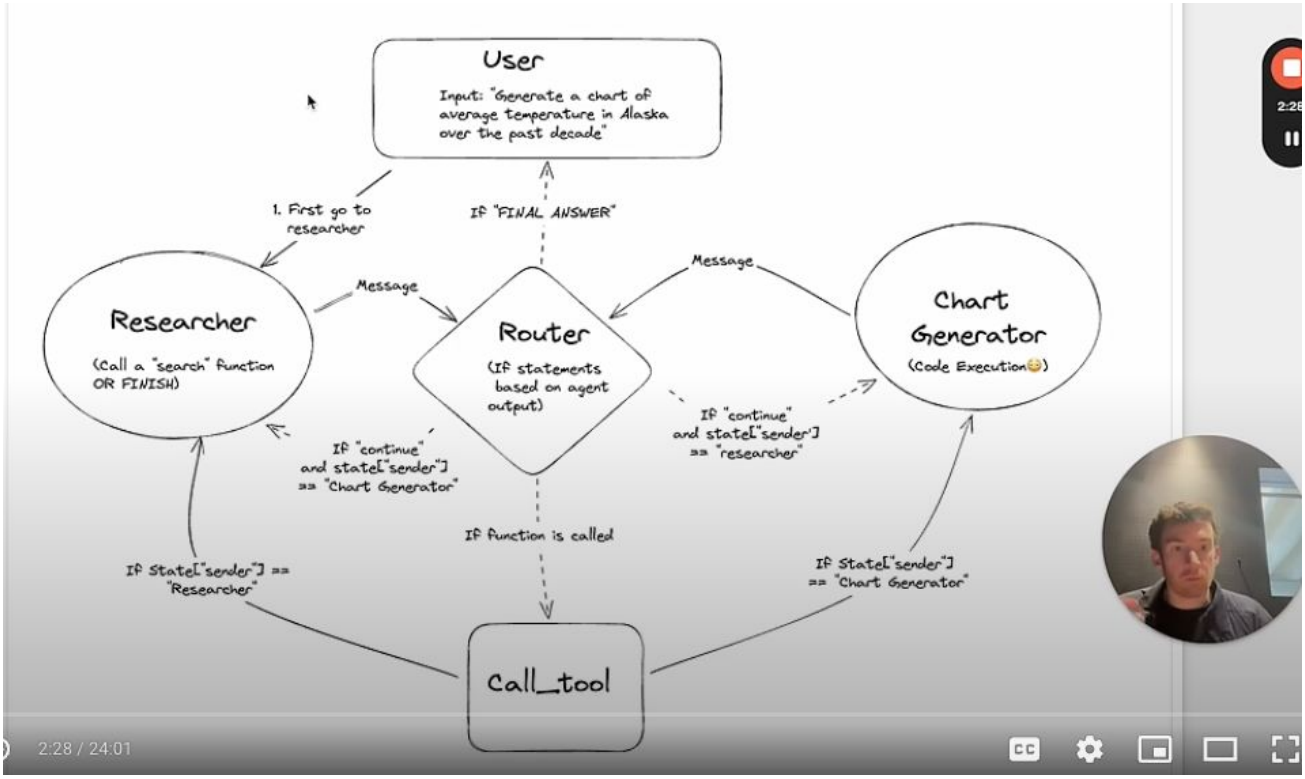
Conclusion

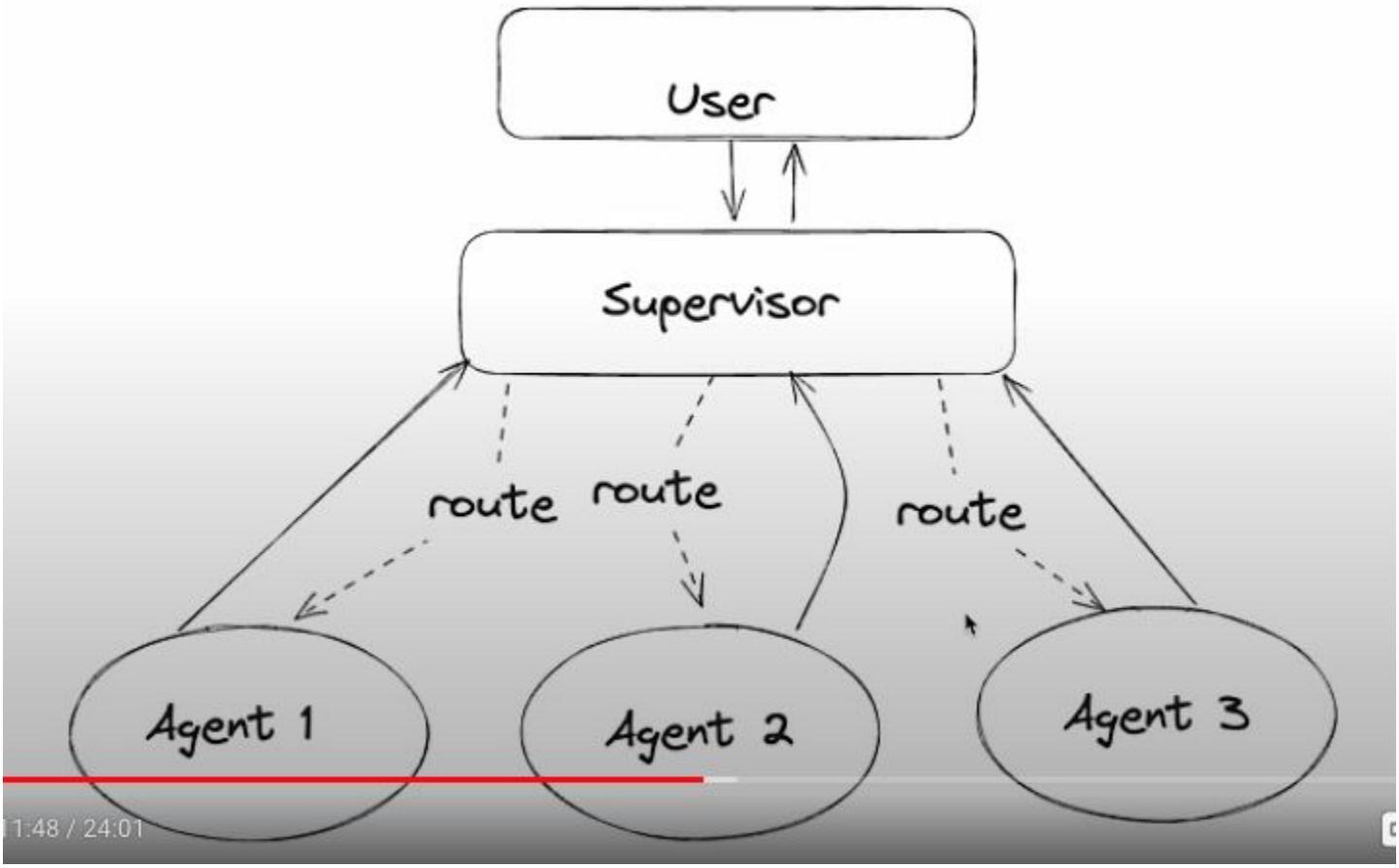


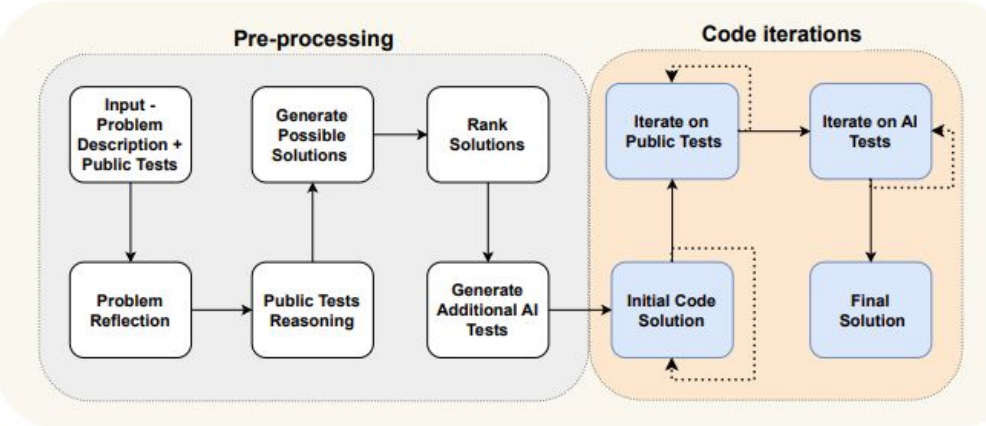
LangChain

DeepLearning.AI

Multi-Agent


User
Input: "Generate a chart of average temperature in Alaska over the past decade"
1. First go to researcher
If "FINAL ANSWER"
Researcher
(Call a "search" function OR FINISH)
Message
Router
(If statements based on agent output)
Message
Chart Generator
(Code Execution)
If "continue" and state["sender"] == "Chart Generator"
If "continue" and state["sender"] == "researcher"
If function is called
If State["sender"] == "Researcher"
If State["sender"] == "Chart Generator"
Call_tool
2:28 / 24:01


User
Supervisor
route
route
route
Agent 1
Agent 2
Agent 3
11:48 / 24:01



(a) The proposed AlphaCodium flow.

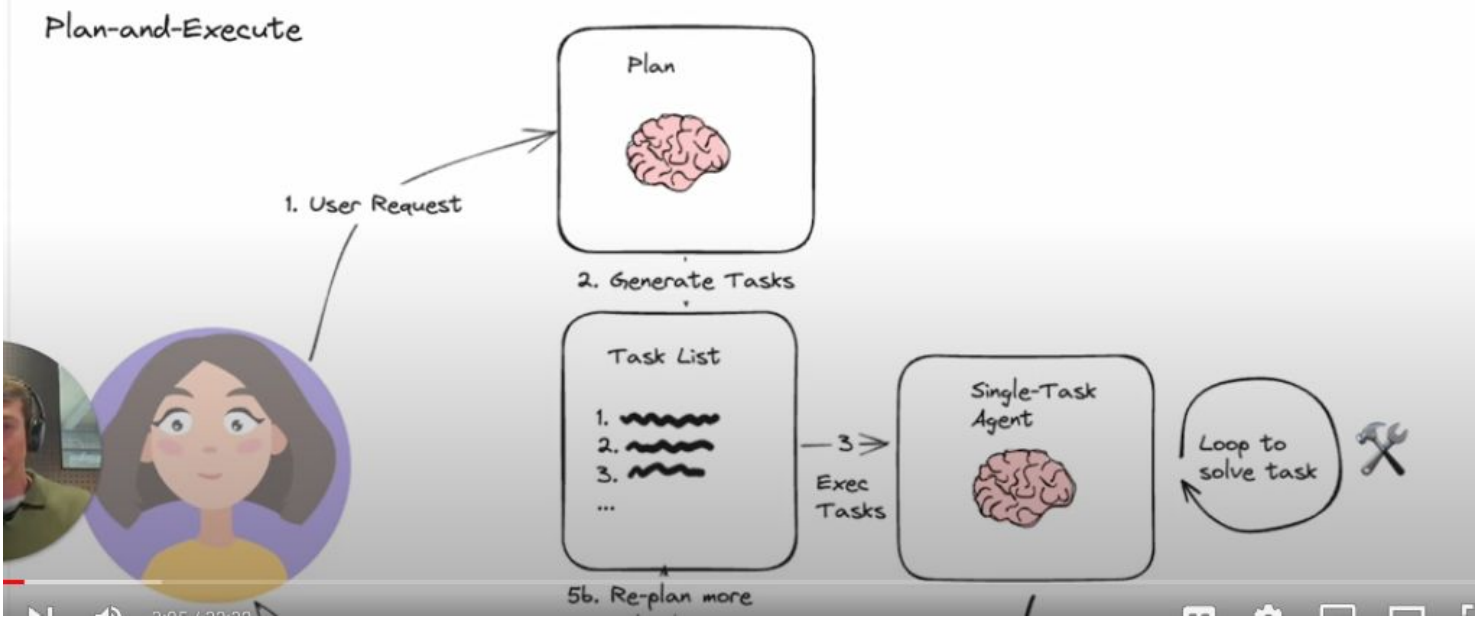
https://arxiv.org/abs/2401.08500

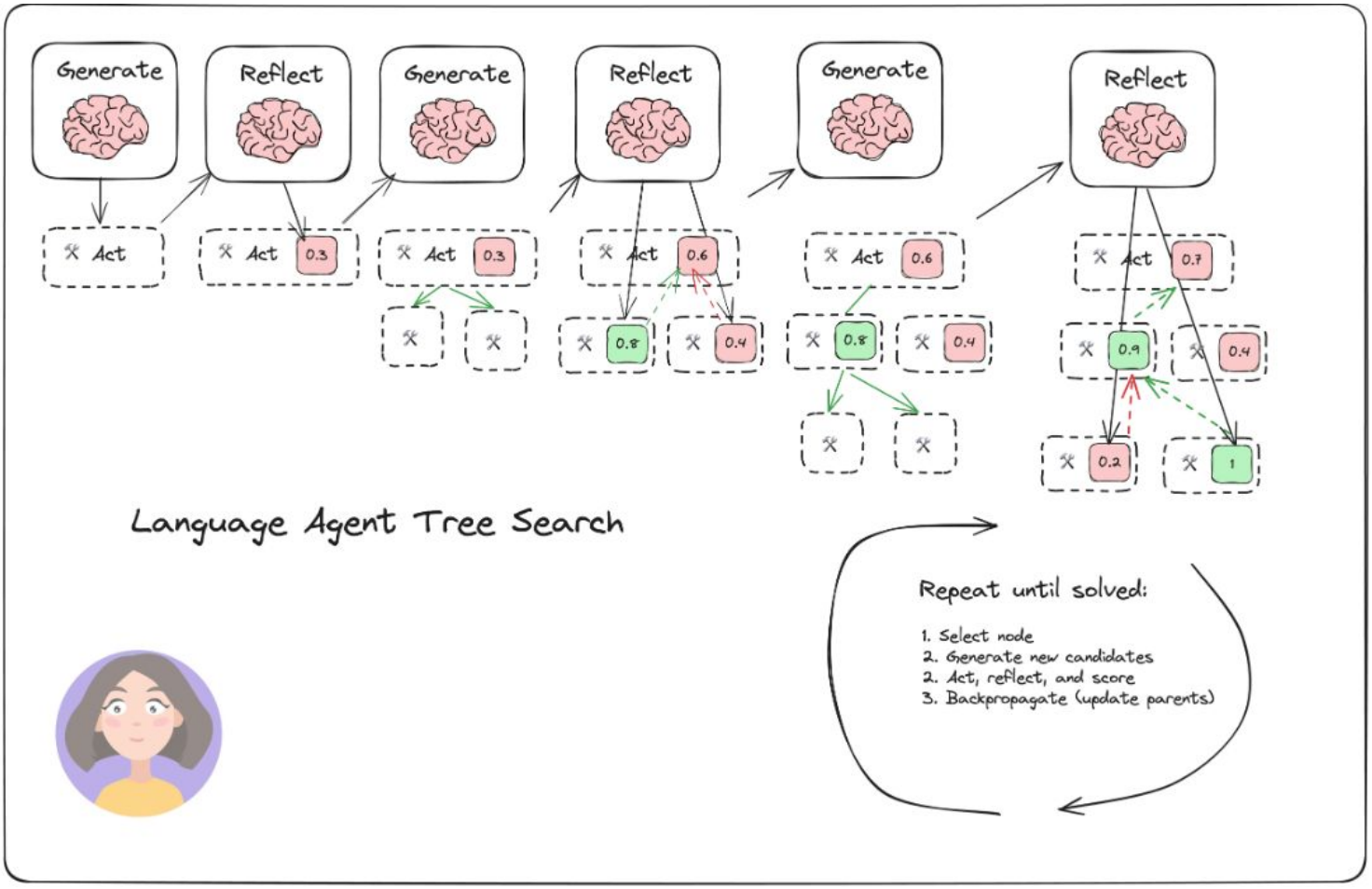
Plan-and-Execute ¶

This notebook shows how to create a "plan-and-execute" style agent. This is heavily inspired by the Plan-and-Solve paper as well as the Baby-AGI project.

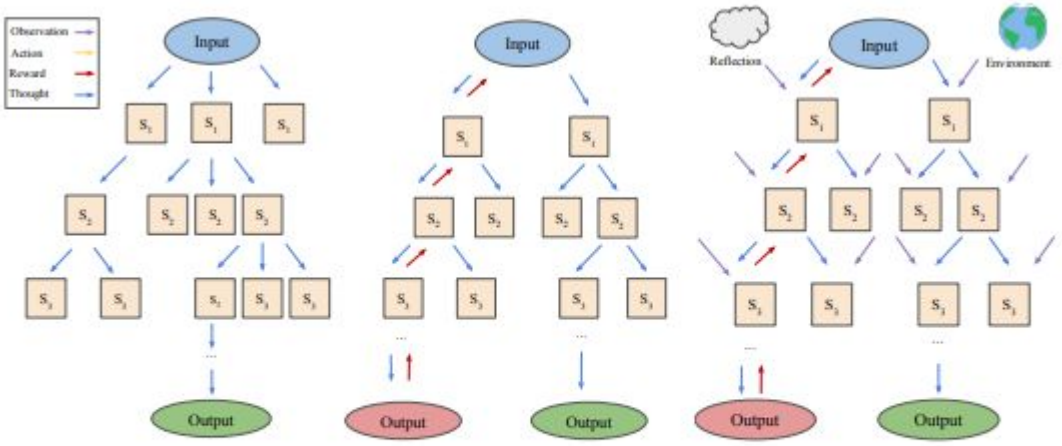
The core idea is to first come up with a multi-step plan, and then go through that plan one item at a time. After accomplishing a particular task, you can then revisit the plan and modify as appropriate.

The general computational graph looks like the following:





https://github.com/langchain-ai/langgraph/blob/main/examples/lats/lats.ipynb



a) Tree-of-Thoughts **b) Reasoning via Planning** **c) Language Agent Tree Search**

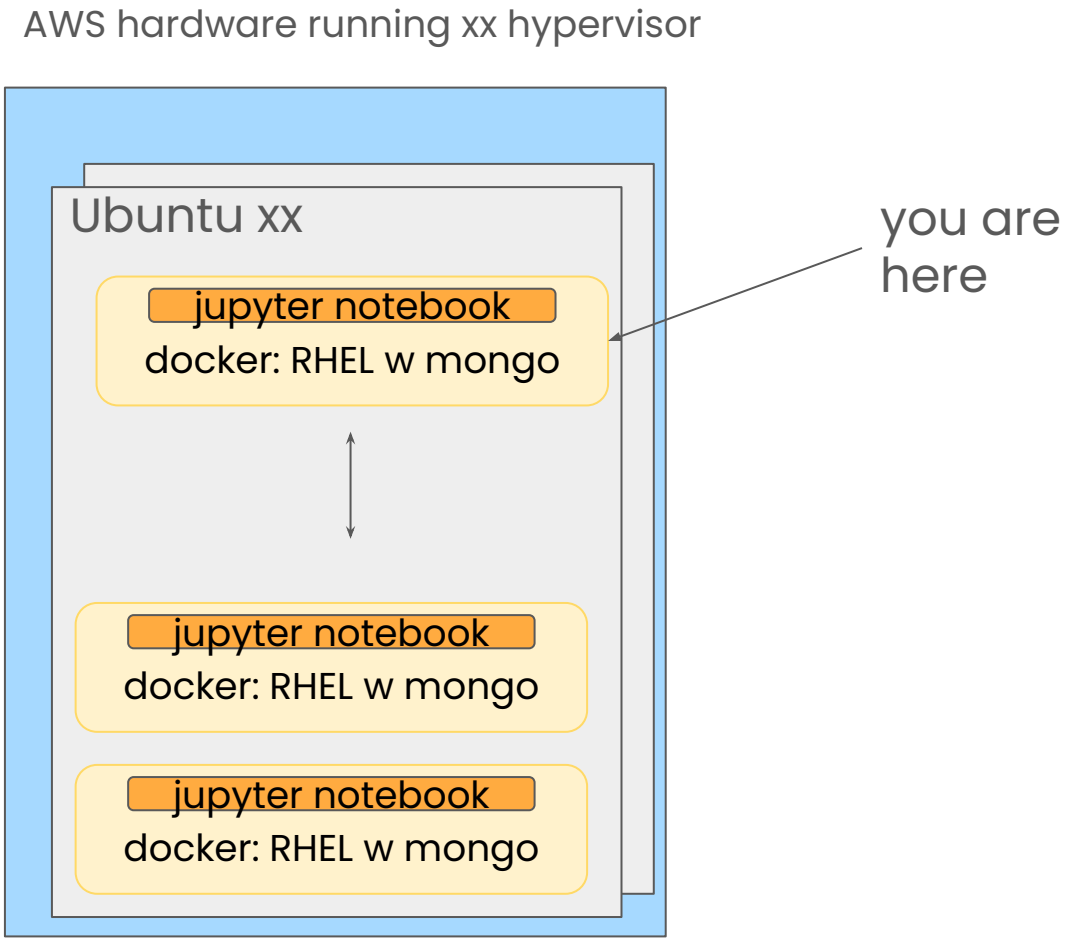
Figure 2: An overview of the differences between LATS and recently proposed LM search algorithms ToT (Yao et al., 2023a) and RAP (Hao et al., 2023). LATS leverages environmental feedback and self-reflection to further adapt search and improve performance.

https://arxiv.org/pdf/2310.04406.pdf

Backup






AWS hardware running xx hypervisor
Ubuntu xx
jupyter notebook
docker: RHEL w mongo
you are here
jupyter notebook
docker: RHEL w mongo
jupyter notebook
docker: RHEL w mongo

Mysteries: Return types

```python
# Define the agent
def run_agent(data):
    agent_outcome = agent_runnable.invoke(data)
    return {"agent_outcome": agent_outcome}
```

dictionary

```python
# Define the function to execute tools
def execute_tools(data):
    # Get the most recent agent_outcome - this is the key added in the `agent` above
    agent_action = data["agent_outcome"]
    output = tool_executor.invoke(agent_action)
    return {"intermediate_steps": [(agent_action, str(output))]}
```

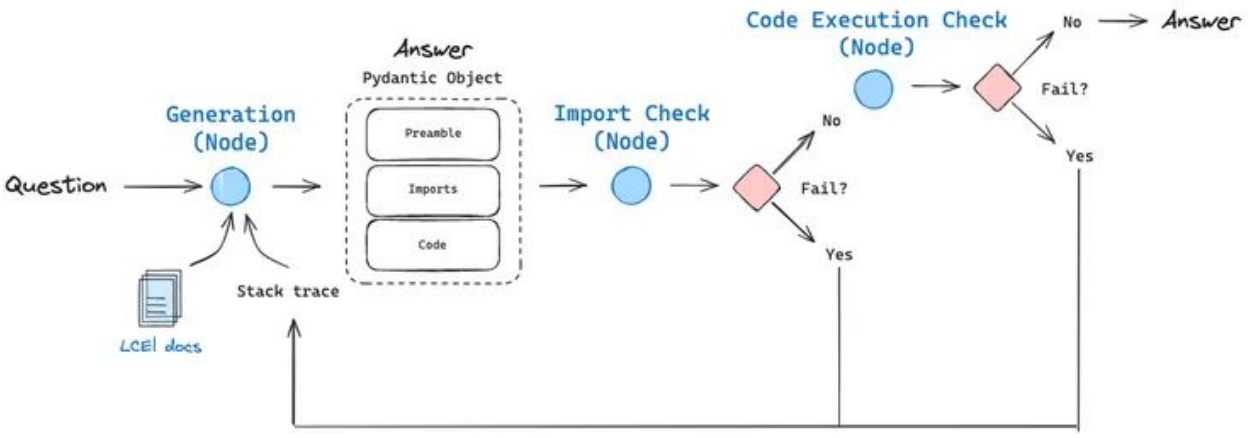
dictionary

```python
# Define logic that will be used to determine which conditional edge to go down
def should_continue(data):
    # If the agent outcome is an AgentFinish, then we return `exit` string
    # This will be used when setting up the graph to define the flow
    if isinstance(data["agent_outcome"], AgentFinish):
        return "end"
    # Otherwise, an AgentAction is returned
    # Here we return `continue` string
    # This will be used when setting up the graph to define the flow
    else:
        return "continue"
```

string

Actually, pretty clear as these nicely match the state, but

- who/what is taking this output and doing something with it?
- and where is it going? (Global data structure?)


Question
Generation
(Node)
Stack trace
LCEL docs
Answer
Pydantic Object
Preamble
Imports
Code
Import Check
(Node)
Fail?
No
Yes
Code Execution Check
(Node)
Fail?
No
Answer
Yes



ReAct Example

https

(1) Hotspot QA

Question: Aside from the Apple Remote, what other device can control the program Apple Remote was originally designed to interact with?

(1d) ReAct (Reason + Act)

Thought 1: I need to search Apple Remote and find the program it was originally designed to interact with.
Act 1: Search[Apple Remote]
Obs 1: The Apple Remote is a remote control introduced in October 2005 by Apple … originally designed to control the Front Row media center program …

Thought 2: Apple Remote was originally designed to control the Front Row media center program. I need to search Front Row next and find what other device can control it.
Act 2: Search[Front Row]
Obs 2: Could not find [Front Row]. Similar: ['Front Row Seat to Earth', 'Front Row Motorsports',' Front Row (software) ', …]

Thought 3: Front Row is not found. I need to search Front Row (software) .
Act 3: Search[Front Row (software)]
Obs 3: Front Row is a discontinued media center software …

Thought 4: Front Row (software) is controlled by an Apple Remote or the keyboard function keys. So the answer is keyboard function keys.
Act 4: Finish[keyboard function keys]

